GNU / Linux

Foundation Training — 3 Days

© Utopios Training

Training Program — Overview

Day	Morning (9h – 12h30)	Afternoon (14h – 17h30)
Day 1	Introduction to GNU/Linux	System Architecture
Day 2	Getting Started & File System	File Management & Shell Scripting
Day 3	Networking	Scripting, Services & Administration

Format: Instructor-led with demos, exercises & hands-on labs
Prerequisites: Basic computer skills — no prior Linux experience required
Environment: Virtual Machine (Ubuntu/Debian) or Docker containers

© Utopios Training

Detailed Program — Day 1

Morning — Introduction to GNU/Linux

- Origins of GNU / Linux — History & Timeline
- Free Software & Open Source — Definitions, Licenses, Organizations
- Linux ecosystem, distributions & market share
- **Demo:** Discovering Linux distributions via containers
- **Exercise:** First steps in the terminal

Afternoon — System Architecture

- System layers: hardware → kernel → libraries → shell → applications
- Linux boot process (BIOS/UEFI → GRUB → Kernel → systemd)
- Kernel architecture & modules
- Graphical layers, desktop environments & shells
- Major distributions — how to choose
- **Lab:** Identify system layers, explore boot & systemd

© Utopios Training

Detailed Program — Day 2

Morning — Getting Started & File System

- Shell basics, configuration files & environment variables
- The PATH variable, wildcards & globbing
- Navigating files & folders, file type identification
- User accounts, groups & permissions
- File System Hierarchy (FHS) & file system types
- **Demo:** User & group management
- **Exercises:** Filesystem exploration
- **Lab:** Complete user management project

© Utopios Training

Detailed Program — Day 2

Afternoon — File Management & Shell Scripting

- File operations, aliases, hard links & symbolic links
- File permissions in depth: chmod, chown, SUID, SGID, sticky bit, ACLs
- Shell scripting foundations: variables, control flow, functions
- sed & awk introduction
- Pipes, redirections & text processing
- **Demo:** Permissions in detail
- **Exercises:** Links & aliases
- **Lab:** Backup script, CSV user creation, log analyzer

© Utopios Training

Detailed Program — Day 3

Morning — Networking

- Networking concepts: IP, TCP/UDP, DNS, DHCP
- OSI & TCP/IP models
- Network configuration & diagnostics under Linux
- DNS resolution, firewall basics (iptables/ufw)
- SSH, SCP, rsync — remote access & file transfer
- **Demo:** Networking between containers
- **Exercise:** Network diagnostics & troubleshooting

© Utopios Training

Detailed Program — Day 3

Afternoon — Scripting, Services & Administration

- Package management (apt / dnf / pacman)
- Standard streams, redirections & advanced pipes
- Cron jobs & task scheduling
- Log management (journalctl, logrotate)
- systemd services & system monitoring
- Disk management basics
- **Demo:** Redirections & pipes
- **Labs:** Package management, system monitoring, disk
- **Final Lab:** Build a server health check script

© Utopios Training

Day 1 — Morning

Introduction to GNU/Linux

© Utopios Training

Agenda — Day 1 Morning

- Origins of GNU / Linux
- Timeline of key milestones
- Free Software & Open Source definitions
- Key organizations and communities
- Licenses in depth
- Overview of open-source operating systems and solutions
- Linux ecosystem and market share

© Utopios Training

Origins of GNU / Linux

The GNU Project (1983)

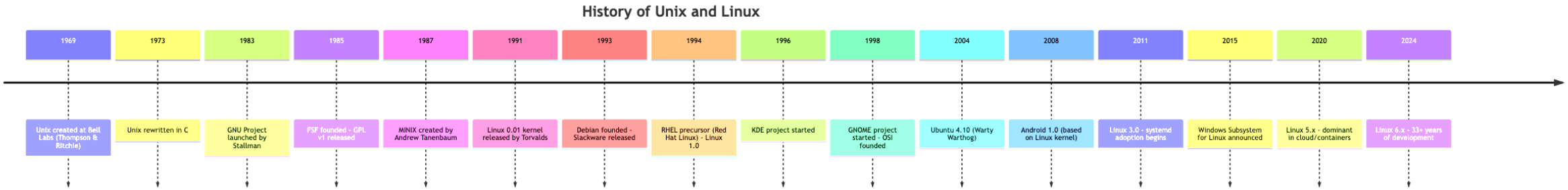
- Founded by **Richard Stallman** at MIT
- Goal: build a completely free Unix-like OS
- Produced core tools: `gcc`, `bash`, `glibc`, `make`...
- Missing piece: a **kernel**

The Linux Kernel (1991)

- Created by **Linus Torvalds**, student in Helsinki
- First announced on the comp.os.minix newsgroup
- Combined with GNU tools → **GNU/Linux** OS

> *"Linux is free software, but also a global engineering achievement."*

Timeline — From Unix to Modern Linux



The Unix Philosophy

Core Principles

- **Do one thing and do it well** — each program handles a single task
- **Write programs that work together** — compose via pipes and text streams
- **Use text streams as the universal interface** — everything is text

Why It Matters Today

```
# Classic Unix pipeline: find the 10 most common words in a file
cat book.txt | tr ' ' '\n' | tr '[:upper:]' '[:lower:]' \
  | sort | uniq -c | sort -rn | head -10
```

“ The philosophy shaped how Linux tools are designed: small, composable, reusable. ”

Free Software vs Open Source

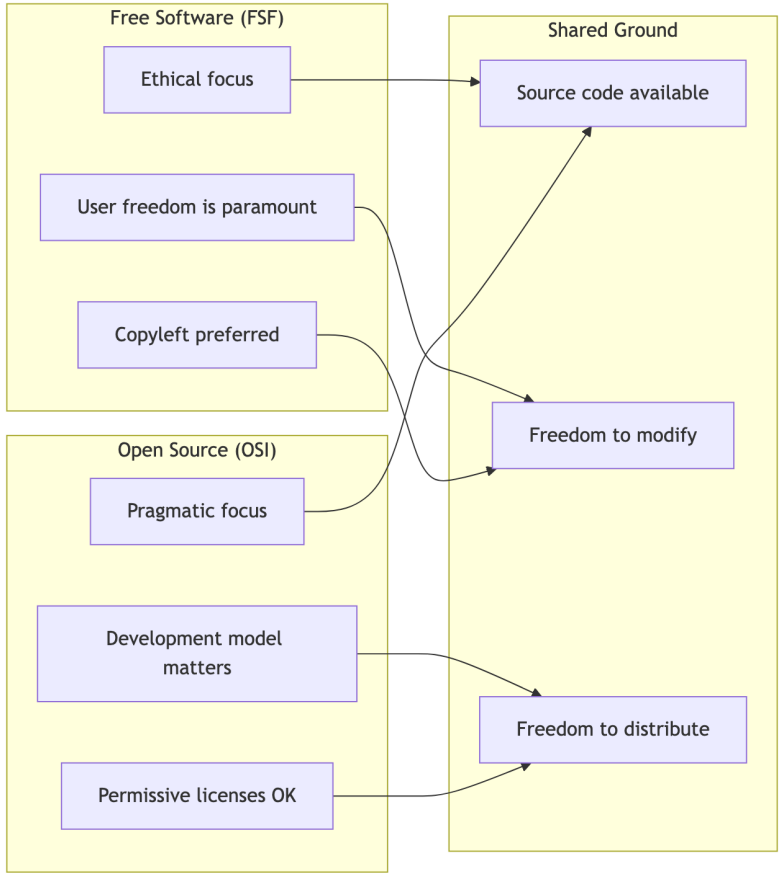
Concept	Definition	Key figure
Free Software	Freedom to run, study, modify, distribute	Richard Stallman / FSF
Open Source	Source code publicly available, focus on pragmatism	Eric Raymond / OSI

The Four Freedoms (GNU/FSF)

0. Freedom to **run** the program
1. Freedom to **study** and modify it
2. Freedom to **redistribute** copies
3. Freedom to **distribute modified** versions

© Utopios Training

Free Software vs Open Source — Comparison



“ **"Free as in freedom, not free as in beer"** — The term "free" refers to liberty, not price. ”

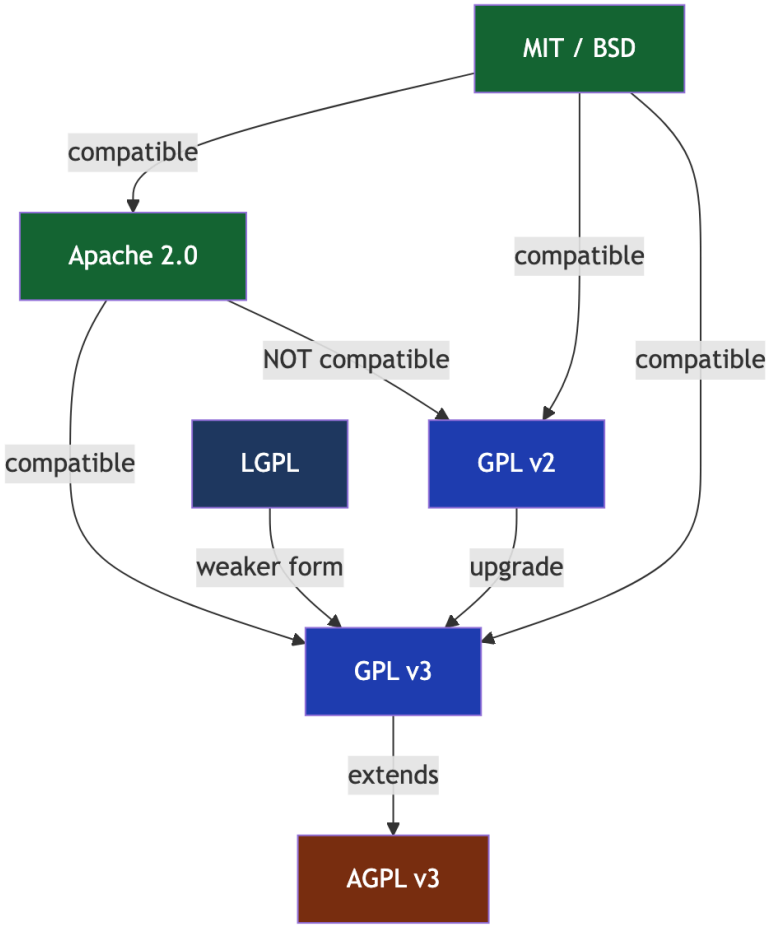
Key Licenses

License	Type	Key Feature
GPL v2 / v3	Copyleft	Derivative works must stay GPL
LGPL	Weak copyleft	Libraries — allows proprietary linking
MIT / BSD	Permissive	Very few restrictions
Apache 2.0	Permissive	Includes patent grant
AGPL	Strong copyleft	Covers network use

“ **Copyleft**: any distributed derivative work must use the same license. ”

© Utopios Training

License Compatibility



“ Permissive code can flow into copyleft projects, but not the reverse. ”

Key Organizations

FSF — Free Software Foundation

- Maintains GNU Project, defends software freedom
- Publishes & enforces GPL licenses

OSI — Open Source Initiative

- Defines the Open Source Definition (OSD)
- Certifies compliant licenses

Linux Foundation

- Hosts the Linux kernel project
- Manages: Kubernetes, Node.js, CNCF, OpenSSF…

Debian Project / Apache Foundation / GNOME Foundation…

© Utopios Training

Overview of Free OS & Solutions

Desktop / General Purpose

`Ubuntu` · `Debian` · `Fedora` · `Arch Linux` · `openSUSE`

Server / Enterprise

`RHEL` · `AlmaLinux` · `Rocky Linux` · `Debian` · `Ubuntu Server`

Embedded / IoT

`OpenWRT` · `Buildroot` · `Yocto Project`

Other Free Solutions

- Office: **LibreOffice**
- Browser: **Firefox**
- Database: **PostgreSQL**, **MariaDB**
- Web server: **Apache**, **Nginx**

© Utopios Training

Linux Ecosystem & Market Share

Where Linux Dominates

Domain	Linux Share	Key Players
Cloud servers	~90%	AWS, GCP, Azure all run Linux
Supercomputers	100%	TOP500 — all run Linux
Mobile	~72%	Android is Linux-based
Embedded / IoT	~60%+	Routers, smart TVs, cars
Web servers	~80%	Apache, Nginx on Linux
Desktop	~4%	Growing steadily (Steam Deck, etc.)
Containers	~99%	Docker, Kubernetes = Linux-native

“ Linux runs on everything from smartwatches to Mars rovers (Ingenuity helicopter). ”

Lab — Day 1 Morning

LAB

Explore Linux History & System Info

```
# 1. Check your distribution
cat /etc/os-release
lsb_release -a 2>/dev/null

# 2. Kernel version and build date
uname -r
uname -v

# 3. System uptime and load
uptime

# 4. List installed GNU tools
ls /usr/bin/gnu* 2>/dev/null
gcc --version 2>/dev/null
bash --version

# 5. Check if your system uses GPL software
dpkg -l | head -20    # Debian/Ubuntu
rpm -qa | head -20    # RHEL/Fedora
```

© Utopios Training

Day 1 — Afternoon

System Architecture

© Utopios Training

Agenda — Day 1 Afternoon

- Foundations of system architecture
- The Linux boot process
- General characteristics of system layers
- Kernel architecture in depth
- Systemd vs SysVinit
- Graphical layers and desktop environments
- Overview of shells
- Main Linux distributions
- Choosing and installing a distribution

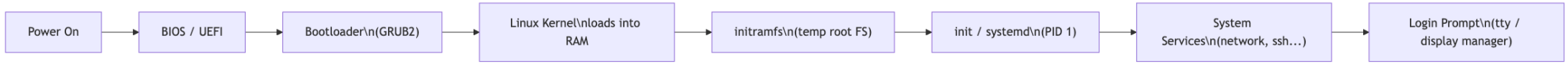
© Utopios Training

System Architecture — Big Picture

```
┌──────────────────────────────────────┐
│          User Applications           │
├──────────────────────────────────────┤
│          Shell / CLI / GUI           │
├──────────────────────────────────────┤
│          System Libraries (glibc)    │
├──────────────────────────────────────┤
│          System Calls Interface      │
├──────────────────────────────────────┤
│          Linux Kernel                │
│   (Process / Memory / FS / Network / IO)│
├──────────────────────────────────────┤
│          Hardware                    │
└──────────────────────────────────────┘
```

© Utopios Training

The Linux Boot Process

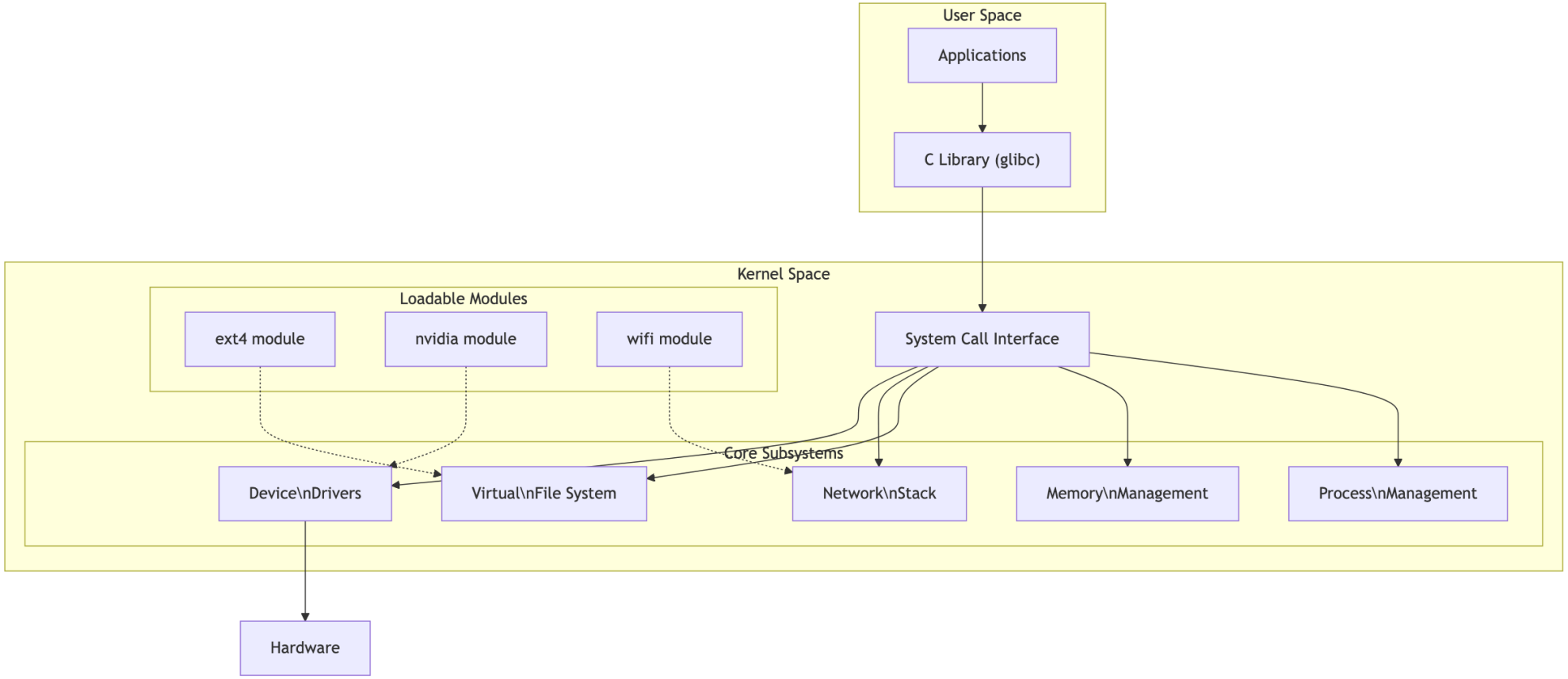


Key Stages

Stage	Role
BIOS/UEFI	Hardware init, finds bootloader
GRUB2	Menu, loads kernel + initramfs
Kernel	Hardware detection, mounts root FS
initramfs	Temporary root with essential drivers
systemd (PID 1)	Starts all services in parallel

© Utopios Training

The Linux Kernel — Architecture



The Linux Kernel — Details

Core Responsibilities

- **Process management**: scheduling, forking, signals
- **Memory management**: virtual memory, paging, swap
- **File system**: VFS abstraction, ext4, xfs, btrfs...
- **Network stack**: TCP/IP, sockets, iptables hooks
- **Device drivers**: hardware abstraction layer
- **Security modules**: SELinux, AppArmor, seccomp

Kernel Types

- **Monolithic** (Linux) — all subsystems in kernel space, but supports **loadable modules**
- **Microkernel** (MINIX, HURD) — minimal kernel, services in user space
- **Hybrid** (Windows NT, macOS XNU) — mix of both

© Utopios Training

Kernel Modules

```
# List loaded modules
lsmod

# Get module info
modinfo ext4
modinfo snd_hda_intel

# Load / unload modules
sudo modprobe vfat           # load module
sudo modprobe -r vfat        # unload module

# Module parameters
modinfo -p iwlwifi           # show parameters
sudo modprobe iwlwifi power_save=1

# Persistent module config
cat /etc/modprobe.d/blacklist.conf
echo "blacklist nouveau" | sudo tee /etc/modprobe.d/blacklist-nouveau.conf
```

“ Linux is monolithic but **modular**: drivers can be loaded/unloaded at runtime without rebooting. ”

Systemd vs SysVinit

Feature	SysVinit	systemd
Init process	Sequential scripts	Parallel unit files
Config location	`/etc/init.d/`	`/etc/systemd/system/`
Service control	`service X start`	`systemctl start X`
Boot speed	Slow (sequential)	Fast (parallel + on-demand)
Dependencies	Numbered scripts	Declarative `After=`, `Requires=`
Logging	Syslog (`/var/log/`)	journald (`journalctl`)
Process tracking	PID files	cgroups
Adoption	Legacy / Devuan	Debian, Ubuntu, RHEL, Fedora, Arch

```
# Systemd targets (equivalent to runlevels)
systemctl get-default                  # current target
systemctl set-default multi-user.target  # text mode
systemctl set-default graphical.target   # GUI mode
```

Graphical Layers

Display Server

- **X11 (X.Org)**: classic, client-server architecture, network-transparent
- **Wayland**: modern replacement, compositor protocol, better security

Desktop Environments

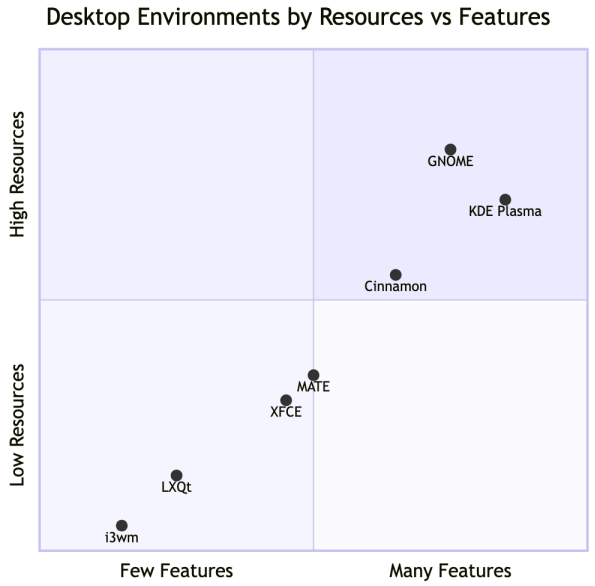
Environment	Toolkit	Style
GNOME	GTK	Modern, clean
KDE Plasma	Qt	Feature-rich
XFCE	GTK	Lightweight
LXQT	Qt	Very lightweight
Cinnamon	GTK	Traditional

Window Managers (standalone)

`i3` · `Openbox` · `bspwm` · `Sway`

© Utopios Training

Desktop Environments — Comparison



Choosing a Desktop

- **Need full features + modern UX?** → GNOME or KDE Plasma
- **Old hardware or VM?** → XFCE or LXQt
- **Keyboard-driven workflow?** → i3, Sway, bspwm
- **Classic GNOME 2 feel?** → MATE or Cinnamon

Overview of Shells

Shell	Description
sh	Original Bourne shell — basic, portable
bash	Bourne Again Shell — standard on most distros
zsh	Extended bash, plugins (Oh My Zsh), auto-complete
fish	User-friendly, syntax highlighting out-of-the-box
dash	Minimal POSIX sh, used for scripts in Debian/Ubuntu
ksh	Korn shell — scripting-focused

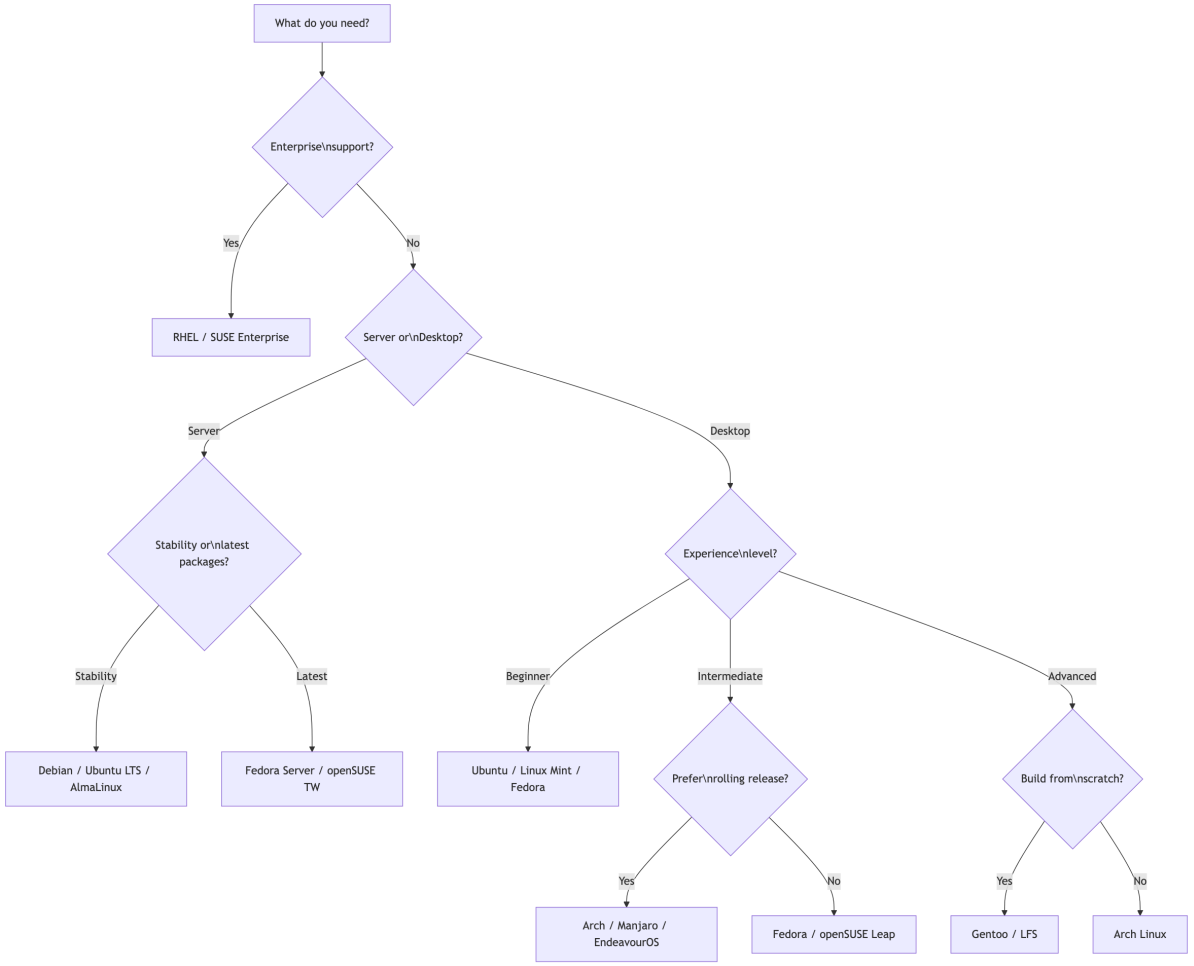
```
echo $SHELL        # Current shell
cat /etc/shells    # All available shells
chsh -s /bin/zsh   # Change default shell
```

© Utopios Training

Major Linux Distributions

```
Debian ──┬── Ubuntu ──┬── Linux Mint
         │            └── Pop!_OS, Kubuntu...
         └── Kali Linux, MX Linux

Red Hat ─┬── RHEL (commercial)
         ├── Fedora (upstream)
         └── AlmaLinux / Rocky Linux (RHEL clones)

Arch ────┬── Arch Linux (rolling)
         └── Manjaro, EndeavourOS

SUSE ────┬── openSUSE Leap (stable)
         └── openSUSE Tumbleweed (rolling)
```

Choosing a Distribution



Distribution Selection — Quick Reference

Use Case	Recommended Distro
Learning / Desktop	Ubuntu, Fedora, Linux Mint
Server / Production	Debian, Ubuntu Server, AlmaLinux
Security / Pentest	Kali Linux, Parrot OS
Minimal / Custom	Arch Linux, Gentoo
Enterprise	RHEL, SUSE Enterprise
Embedded	Buildroot, Yocto

Key criteria: stability, community, package ecosystem, support cycle, hardware compatibility

© Utopios Training

Lab — Day 1 Afternoon

LAB

Identify System Layers

```
# Distribution information
cat /etc/os-release
lsb_release -a

# Kernel version
uname -r
uname -a

# Architecture
arch
uname -m

# Loaded kernel modules
lsmod | head -20

# Running display server
echo $XDG_SESSION_TYPE   # x11 or wayland
```

Lab — Boot & Systemd Exploration

LAB

```
# 1. Check init system
ps -p 1 -o comm=          # Should show "systemd"
systemctl --version

# 2. Analyze boot time
systemd-analyze
systemd-analyze blame | head -15   # Slowest services

# 3. View systemd targets
systemctl list-units --type=target

# 4. Inspect a service unit file
systemctl cat ssh              # or sshd on RHEL
systemctl status ssh

# 5. View boot log
journalctl -b -p err           # errors from current boot
journalctl --list-boots         # list previous boots

# 6. GRUB config (read only!)
cat /etc/default/grub
```

Day 2 — Morning

Getting Started & File System

© Utopios Training

Agenda — Day 2 Morning

- Shell initiation and environment
- Shell configuration files and loading order
- Environment variables
- Graphical interface introduction
- Navigating files and folders
- Wildcards and globbing
- File type identification
- User accounts and administrators
- Lab: `sudo`, `su`, user creation
- File system tree (FHS)
- File system types

© Utopios Training

Shell Basics

Anatomy of a Command

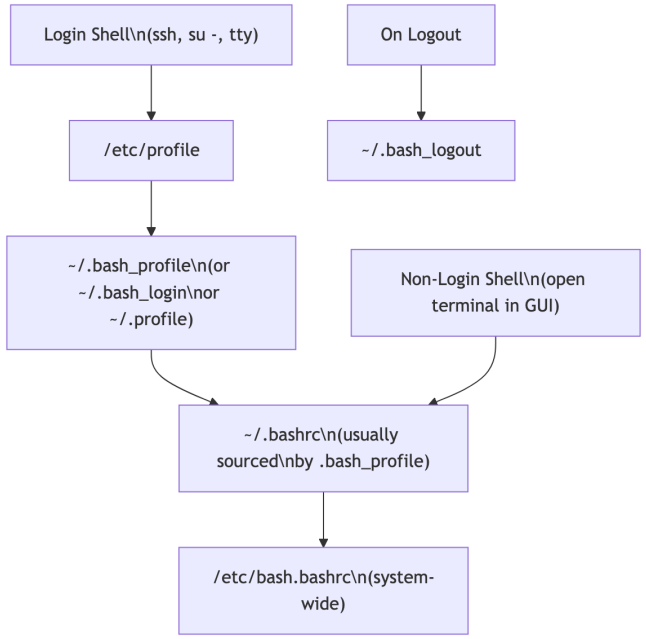
```
command  [options]  [arguments]
  ls       -la      /home/user
```

- **Command**: executable or built-in
- **Options**: flags that modify behaviour (`-l`, `--all`)
- **Arguments**: targets the command acts on

First Commands

```
whoami          # current user
hostname        # machine name
pwd             # current directory
date            # current date/time
uptime          # system uptime
```

Shell Configuration Files



Loading Order Summary

Shell Type	Files Loaded
Login (ssh, tty)	`/etc/profile` → `~/.bash_profile` → `~/.bashrc`
Non-login (GUI terminal)	`/etc/bash.bashrc` → `~/.bashrc`
Script (`#!/bin/bash`)	None (unless `--login` flag or sourced)

Environment Variables

```
# View all variables
env                       # environment variables only
set                       # all variables (including local)
printenv PATH             # specific variable

# Important variables
echo $HOME                # user home directory
echo $PATH                # executable search path
echo $USER                # current username
echo $SHELL               # default shell
echo $LANG                # system locale
echo $EDITOR              # default text editor
echo $PS1                 # prompt format

# Set variables
export MY_VAR="value"     # available to child processes
MY_LOCAL="value"          # local to current shell only

# Persist in ~/.bashrc
echo 'export MY_VAR="value"' >> ~/.bashrc
source ~/.bashrc          # reload without new shell
```

© Utopios Training

The PATH Variable

```
# View PATH
echo $PATH
# /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin

# Add directory to PATH
export PATH="$HOME/bin:$PATH"       # prepend (priority)
export PATH="$PATH:/opt/myapp/bin"  # append

# Where is a command?
which python3       # first match in PATH
type -a python3     # all matches
command -v python3  # POSIX-compliant check
```

How the Shell Finds Commands

```
User types "ls" → Shell searches PATH left to right:
  /usr/local/bin/ls ? → NO
  /usr/bin/ls ? → YES → Execute!
```

“ If a command is "not found", either install it or add its directory to `$PATH`. ”

Navigating Files & Folders

```
ls              # list directory contents
ls -la          # long format, including hidden files
cd /var/log     # change directory (absolute)
cd ../..        # go up two levels (relative)
cd ~            # go to home directory
cd -            # go to previous directory

mkdir mydir               # create directory
mkdir -p a/b/c            # create nested dirs
rmdir emptydir            # remove empty directory

cp source dest            # copy file
cp -r source/ dest/       # copy directory recursively
mv source dest            # move / rename
rm file.txt               # remove file
rm -rf directory/         # remove directory recursively
```

Wildcards & Globbing

```
# Glob patterns (expanded by the shell, not the command)
ls *.txt              # all .txt files
ls file?.log          # file1.log, fileA.log (? = one char)
ls [abc]*             # files starting with a, b, or c
ls [!0-9]*            # files NOT starting with a digit
ls {*.txt,*.md}       # .txt and .md files (brace expansion)

# Practical examples
cp /etc/*.conf ~/backup/
rm /tmp/test_[0-9][0-9].log
mv report_{jan,feb,mar}.pdf archive/
```

Globbing Reference

Pattern	Meaning	Example
`*`	Any number of chars	`*.log`
`?`	Exactly one char	`file?.txt`
`[abc]`	One of a, b, or c	`[Rr]eadme`
`[a-z]`	Range a to z	`[0-9]*`
`[!x]`	Not x	`[!.]*` (no hidden)
`{a,b}`	a or b (brace exp.)	`file.{txt,md}`

File Type Identification

```
# The file command — identify file types
file document.pdf        # PDF document, version 1.4
file script.sh           # Bourne-Again shell script
file image.png           # PNG image data, 800x600
file /bin/ls             # ELF 64-bit LSB executable
file /dev/sda            # block special

# How it works: magic numbers (first bytes of file)
xxd -l 16 image.png     # shows PNG magic: 89 50 4e 47
xxd -l 16 document.pdf  # shows PDF magic: 25 50 44 46 (%PDF)

# stat — detailed file metadata
stat file.txt
# Shows: size, blocks, inode, permissions, timestamps
#   Access (atime), Modify (mtime), Change (ctime)

# Identify file with ls -F
ls -F    # / = dir, * = exec, @ = symlink, | = pipe
```

User Accounts & Permissions

Account Types

- **root**: superuser, UID 0, unlimited privileges
- **regular user**: limited to own files and permitted resources
- **system users**: services (daemon, www-data, nobody)

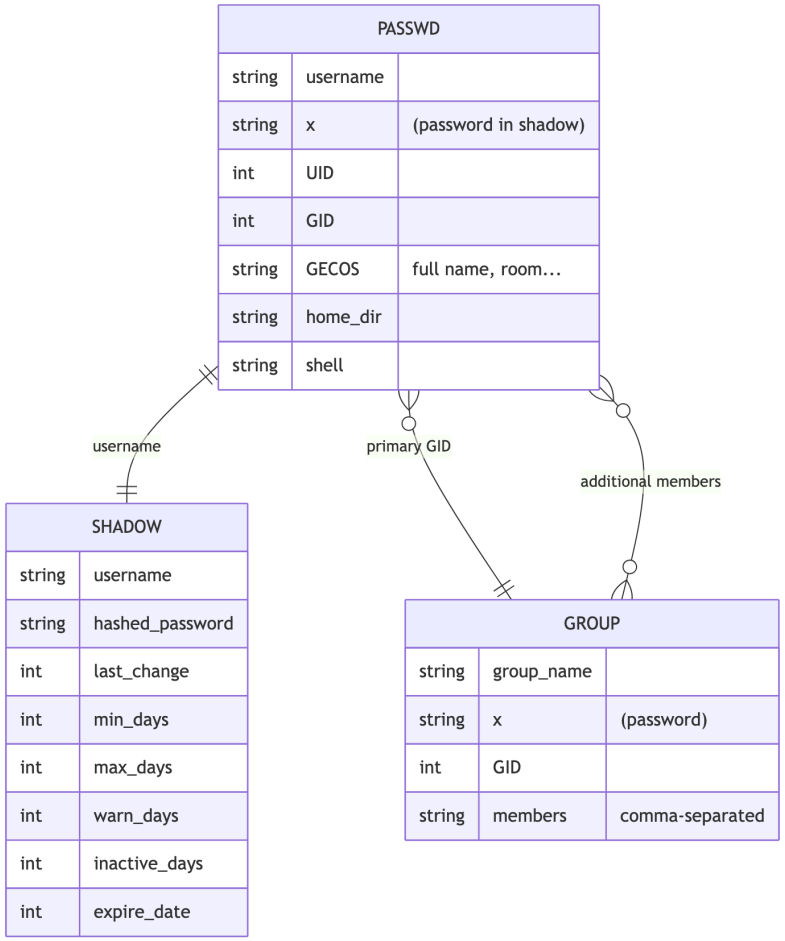
Key Files

```
cat /etc/passwd   # user database
cat /etc/shadow   # hashed passwords (root only)
cat /etc/group    # group database
```

Switching Users

```
su -              # switch to root (needs root password)
sudo command      # run command as root (needs user's password)
sudo -i           # open root shell via sudo
```

User Database Files — Relationships



Understanding /etc/passwd and /etc/shadow

```
# /etc/passwd — one line per user (7 fields, colon-separated)
# username:x:UID:GID:GECOS:home:shell
root:x:0:0:root:/root:/bin/bash
alice:x:1001:1001:Alice Smith:/home/alice:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

# /etc/shadow — password hashes (root-readable only)
# username:hash:lastchg:min:max:warn:inactive:expire
alice:$6$salt$hash...:19500:0:99999:7:::

# /etc/group — group database
# groupname:x:GID:members
devs:x:1002:alice,bob

# Password hash algorithms
# $1$ = MD5    $5$ = SHA-256    $6$ = SHA-512    $y$ = yescrypt
```

User Management

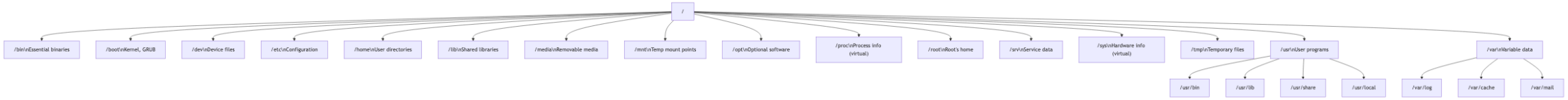
```
# Create user
sudo useradd -m -s /bin/bash alice
sudo passwd alice

# Modify user
sudo usermod -aG sudo alice     # add to sudo group
sudo usermod -l newname alice   # rename

# Delete user
sudo userdel -r alice           # -r removes home dir

# View user info
id alice
groups alice
finger alice      # if installed
```

Lab — User Accounts

LAB

```
# 1. Check current user context
whoami && id

# 2. Use sudo
sudo apt update   # (Debian/Ubuntu)

# 3. Switch user
su - root         # (requires root password)
exit              # return to previous user

# 4. Create a user
sudo useradd -m -s /bin/bash trainee
sudo passwd trainee

# 5. Verify
cat /etc/passwd | grep trainee
ls /home/
```

File System Hierarchy (FHS)



FHS — Text Reference

```
/                   Root
├── bin/            Essential user binaries (ls, cp, bash)
├── boot/           Kernel, bootloader (GRUB)
├── dev/            Device files (sda, tty, null)
├── etc/            System configuration files
├── home/           User home directories
├── lib/            Essential shared libraries
├── media/          Mount points for removable media
├── mnt/            Temporary mount points
├── opt/            Optional / third-party software
├── proc/           Virtual FS — kernel & process info
├── root/           Root user home
├── srv/            Service data (web, ftp)
├── sys/            Virtual FS — hardware/kernel info
├── tmp/            Temporary files (cleared on reboot)
├── usr/            Secondary hierarchy (user programs)
└── var/            Variable data (logs, spool, cache)
```

File System Types

FS Type	Use Case
ext4	Default Linux FS, journaled, stable
xfs	High-performance, large files, RHEL default
btrfs	Snapshots, subvolumes, checksums
tmpfs	RAM-based, `/tmp`, `/run`
nfs	Network file system
fat32 / exFAT	USB, cross-platform
swap	Virtual memory extension

```
df -hT              # disk usage with FS type
lsblk -f            # block devices and FS types
mount | column -t   # currently mounted FS
cat /proc/mounts    # kernel mount table
```

Lab — File System

LAB

```
# 1. View partitioning
lsblk
sudo fdisk -l

# 2. File system types and usage
df -hT
cat /etc/fstab

# 3. Identify mounts
mount | grep "^/dev"

# 4. Inspect /proc virtual FS
cat /proc/cpuinfo
cat /proc/meminfo
ls /proc/$$         # current process dir
```

Lab — Shell Configuration & Environment

LAB

```
# 1. Identify your shell and config files
echo $SHELL
ls -la ~/.bash* ~/.profile 2>/dev/null

# 2. View and modify PATH
echo $PATH
export PATH="$PATH:$HOME/scripts"
echo $PATH   # verify

# 3. Create a persistent alias
echo 'alias ll="ls -la --color"' >> ~/.bashrc
echo 'alias ports="ss -tulnp"' >> ~/.bashrc
source ~/.bashrc
ll

# 4. Set a custom environment variable
echo 'export EDITOR=vim' >> ~/.bashrc
source ~/.bashrc
echo $EDITOR

# 5. Explore the file command
file /etc/passwd /bin/ls /dev/null /tmp
```

© Utopios Training

Day 2 — Afternoon

File Management & Shell Scripting

© Utopios Training

Agenda — Day 2 Afternoon

- Directory and file management commands
- Reading and modifying file content
- Aliases
- Hard links and symbolic links
- File permissions and ownership
- ACLs (Access Control Lists)
- Shell scripting foundations
- Advanced scripting: case, arrays, exit codes
- sed and awk introduction
- Pipes, redirections, grep and regex
- Text processing pipelines
- Main text editors and applications
- Process, daemon, and service intro

© Utopios Training

File Operations

```
# View file content
cat file.txt                # print entire file
less file.txt               # paged viewer (q to quit)
head -n 20 file.txt         # first 20 lines
tail -n 20 file.txt         # last 20 lines
tail -f /var/log/syslog     # live follow

# Search in files
grep "error" file.txt
grep -r "pattern" /var/log/ # recursive
grep -i "ERROR" file.txt    # case insensitive

# Find files
find /home -name "*.conf"
find /var -size +10M -type f
find . -mtime -7            # modified last 7 days
```

Aliases & Links

Aliases

```
alias ll='ls -la'
alias gs='git status'
alias ..='cd ..'
unalias ll
# Persist in ~/.bashrc or ~/.bash_aliases
```

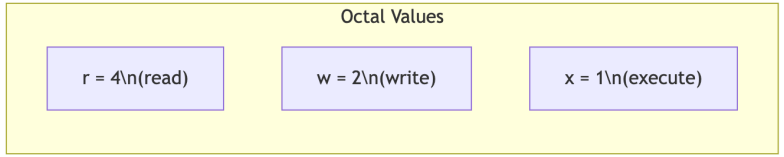
Hard Links vs Symbolic Links

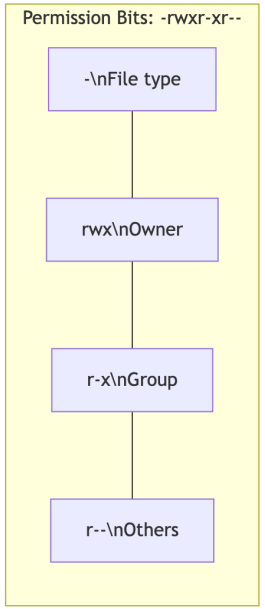
```
ln target hardlink        # hard link — same inode
ln -s target symlink      # symbolic link — pointer

ls -li                    # show inode numbers
readlink symlink          # show target of symlink
```

	Hard Link	Symbolic Link
Cross-filesystem	No	Yes
Works for dirs	No	Yes
Survives target deletion	Yes	No

© Utopios Training

File Permissions





File Permissions

```
-rwxr-xr-- 1 alice devs 4096 Apr 5 10:00 script.sh
│├──┤├──┤├──┤
│owner group others
│
└─ type: - file | d dir | l symlink | c char | b block
```

Perm	File	Directory
r (4)	Read content	List contents
w (2)	Modify content	Create/delete files
x (1)	Execute	Enter directory

Changing Permissions & Ownership

```
chmod 755 script.sh       # octal
chmod u+x,g-w script.sh  # symbolic
chown alice:devs file     # change owner:group
chown -R alice /var/app   # recursive
```

Octal Permission Reference

Octal	Binary	Permissions	Meaning
7	111	rwx	Full
6	110	rw-	Read + Write
5	101	r-x	Read + Execute
4	100	r--	Read only
3	011	-wx	Write + Execute
0	000	---	None

“ Common patterns: `755` (scripts, dirs), `644` (regular files), `600` (private), `700` (private dirs) ”

Special Permissions & umask

```
# SUID (4) — run as file owner
chmod u+s /usr/bin/passwd

# SGID (2) — run as group / inherit group in dir
chmod g+s /shared/

# Sticky bit (1) — only owner can delete in dir
chmod +t /tmp/

# umask — default permission mask
umask             # show current (e.g., 022)
umask 027         # new files: 640, new dirs: 750
```

ACLs — Access Control Lists

```
# ACLs provide fine-grained permissions beyond owner/group/other

# View ACLs
getfacl file.txt

# Set ACL — give user bob read+write on a file
setfacl -m u:bob:rw file.txt

# Set ACL — give group devs read on a directory (recursive)
setfacl -R -m g:devs:r /shared/

# Set default ACL (inherited by new files in dir)
setfacl -d -m g:devs:rwx /shared/

# Remove specific ACL
setfacl -x u:bob file.txt

# Remove all ACLs
setfacl -b file.txt
```

“ When a file has ACLs, `ls -l` shows a `+` after permissions: `-rw-r--r--+` ”

Lab — Permissions & ACLs

LAB

```
# 1. Create structure
mkdir -p /tmp/project/{public,private}
touch /tmp/project/public/readme.txt
touch /tmp/project/private/secret.conf

# 2. Set permissions
chmod 755 /tmp/project/public
chmod 700 /tmp/project/private
chmod 640 /tmp/project/private/secret.conf

# 3. Verify
ls -la /tmp/project/
ls -la /tmp/project/private/

# 4. Test with another user
sudo su - trainee
cat /tmp/project/private/secret.conf  # should fail

# 5. ACL exercise (as your user)
setfacl -m u:trainee:r /tmp/project/private/secret.conf
getfacl /tmp/project/private/secret.conf
# Now trainee CAN read it despite 640 permissions
```

Shell Scripting — Foundations

```
#!/bin/bash
# First line: shebang — tells the OS which interpreter to use

# Variables
NAME="Alice"
echo "Hello, $NAME"
echo "You are: $(whoami)"

# Quoting rules
echo '$NAME'       # literal: $NAME (single = no expansion)
echo "$NAME"       # expanded: Alice (double = expansion)
echo "Today: $(date +%F)"

# Command result
RESULT=$(ls -1 | wc -l)
echo "Files: $RESULT"
```

Control Flow

```
# Conditionals
if [ "$USER" = "root" ]; then
  echo "Running as root"
elif [ "$USER" = "alice" ]; then
  echo "Hello Alice"
else
  echo "Unknown user"
fi

# For loop
for f in /etc/*.conf; do
  echo "Found: $f"
done

# While loop
COUNT=1
while [ $COUNT -le 5 ]; do
  echo "Iteration $COUNT"
  ((COUNT++))
done
```

Case Statements & Select

```
# case — pattern matching (cleaner than if/elif chains)
case "$1" in
  start)
    echo "Starting service..."
    ;;
  stop)
    echo "Stopping service..."
    ;;
  restart)
    echo "Restarting..."
    ;;
  status)
    echo "Checking status..."
    ;;
  *)
    echo "Usage: $0 {start|stop|restart|status}"
    exit 1
    ;;
esac

# select — interactive menu
select OPTION in "Backup" "Restore" "Quit"; do
  case $OPTION in
    Backup)  echo "Backing up..."; break ;;
    Restore) echo "Restoring..."; break ;;
    Quit)    exit 0 ;;
  esac
done
```

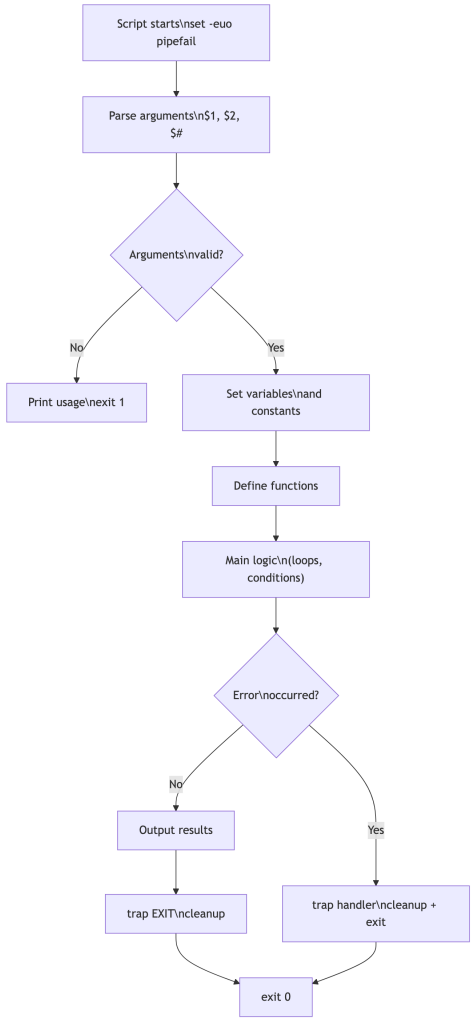
Arrays & String Manipulation

```
# Indexed arrays
FRUITS=("apple" "banana" "cherry")
echo ${FRUITS[0]}          # apple
echo ${FRUITS[@]}          # all elements
echo ${#FRUITS[@]}         # count: 3
FRUITS+=("date")           # append

# Loop over array
for fruit in "${FRUITS[@]}"; do
  echo "Fruit: $fruit"
done

# String manipulation
STR="Hello World"
echo ${#STR}               # length: 11
echo ${STR,,}              # lowercase: hello world
echo ${STR^^}              # uppercase: HELLO WORLD
echo ${STR:0:5}            # substring: Hello
echo ${STR/World/Linux}    # replace: Hello Linux

# Filename manipulation
FILE="/var/log/syslog.gz"
echo ${FILE%.gz}           # remove suffix: /var/log/syslog
echo ${FILE##*/}           # basename: syslog.gz
echo ${FILE%/*}            # dirname: /var/log
```

Exit Codes & Error Handling

```
# Every command returns an exit code (0 = success, non-zero = error)
ls /tmp
echo $?    # 0 (success)

ls /nonexistent
echo $?    # 2 (error)

# Use exit codes in scripts
#!/bin/bash
set -euo pipefail   # strict mode: exit on error, unset vars, pipe fail

# -e: exit immediately on error
# -u: error on unset variables
# -o pipefail: pipe fails if any command fails

# Trap for cleanup
cleanup() {
  echo "Cleaning up..."
  rm -f /tmp/mylock
}
trap cleanup EXIT   # runs on script exit (success or failure)
trap cleanup ERR    # runs on error

# Logical operators
command1 && command2    # run command2 only if command1 succeeds
command1 || command2    # run command2 only if command1 fails
```

Script Execution Flow



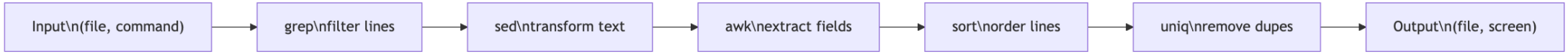
Introduction to sed

```
# sed = Stream EDitor — transforms text line by line

# Substitute (s command) — like find/replace
sed 's/old/new/' file.txt           # first match per line
sed 's/old/new/g' file.txt          # all matches (global)
sed -i 's/old/new/g' file.txt      # in-place edit

# Delete lines
sed '/^#/d' file.txt               # delete comment lines
sed '/^$/d' file.txt               # delete empty lines
sed '1,5d' file.txt                # delete lines 1-5

# Print specific lines
sed -n '10,20p' file.txt           # print lines 10-20
sed -n '/error/p' file.txt         # print lines matching pattern

# Insert / Append
sed '3i\New line before line 3' file.txt
sed '3a\New line after line 3' file.txt

# Multiple operations
sed -e 's/foo/bar/g' -e '/^$/d' file.txt
```

Introduction to awk

```
# awk = Pattern scanning + processing language
# Structure: awk 'pattern { action }' file

# Print specific columns (default separator: whitespace)
awk '{print $1, $3}' file.txt
awk -F: '{print $1, $7}' /etc/passwd    # custom separator

# Filter by pattern
awk '/error/ {print $0}' log.txt
awk '$3 > 100 {print $1, $3}' data.txt  # numeric comparison

# Built-in variables
# NR = line number, NF = number of fields, FS = field separator
awk '{print NR, $0}' file.txt           # add line numbers
awk -F: 'NF == 7 {print $1}' /etc/passwd

# Calculations
awk '{sum += $1} END {print "Total:", sum}' numbers.txt
awk '{sum += $1; n++} END {print "Avg:", sum/n}' numbers.txt

# Formatted output
awk -F: '{printf "%-15s UID: %s\n", $1, $3}' /etc/passwd
```

Text Processing Pipeline



Real-World Example

```
# Find top 10 IP addresses hitting your web server
cat /var/log/apache2/access.log \
  | awk '{print $1}' \
  | sort \
  | uniq -c \
  | sort -rn \
  | head -10

# Count errors by type in syslog
grep -i "error" /var/log/syslog \
  | sed 's/.*error[: ]*//' \
  | sort | uniq -c | sort -rn | head
```

Pipes, Redirections & Variables

```
# Redirections
command > output.txt     # stdout to file (overwrite)
command >> output.txt    # stdout to file (append)
command 2> errors.txt    # stderr to file
command &> all.txt       # stdout + stderr

# Input
command < input.txt
command << EOF           # here-doc
line1
EOF

# Pipes
ls -la | grep "^d" | wc -l      # count directories
ps aux | sort -k3 -rn | head    # top CPU processes
cat /etc/passwd | cut -d: -f1   # list usernames
```

grep & Regular Expressions

```
grep "^root" /etc/passwd        # lines starting with root
grep "sh$" /etc/passwd          # lines ending with sh
grep -E "[0-9]{3}" file         # 3+ digits (ERE)
grep -v "^#" /etc/ssh/sshd_config  # exclude comments

# Key regex metacharacters
# . = any char   * = zero or more   + = one or more (ERE)
# ^ = start      $ = end            [] = char class
# \d = digit     \w = word char     \s = space
# | = OR (ERE)   () = group (ERE)

# Practical examples
grep -E "^[A-Z]" words.txt
grep -iE "(error|warn|crit)" /var/log/syslog
grep -rn "TODO" ~/projects/
```

Text Editors Overview

Editor	Style	Best For
nano	Simple, menu-driven	Quick edits, beginners
vim / vi	Modal, powerful	Servers, scripting
emacs	Extensible	Power users
gedit	GUI (GNOME)	Desktop editing
VSCode	GUI + terminal	Development

vim Quick Reference

```
i       → insert mode       Esc → normal mode
:w      → save              :q  → quit
:wq     → save & quit       :q! → force quit
dd      → delete line       yy  → copy line
p       → paste             /word → search
```

Processes, Daemons & Services

```
# Process management
ps aux                    # all processes
top / htop                # interactive process viewer
kill PID                  # send SIGTERM
kill -9 PID               # send SIGKILL (force)
killall nginx             # by name

# Background / foreground
command &                 # run in background
jobs                      # list background jobs
fg %1                     # bring job 1 to foreground
bg %1                     # resume job in background

# Services (systemd)
systemctl status nginx
systemctl start / stop / restart nginx
systemctl enable nginx    # start at boot
journalctl -u nginx -f    # follow service logs
```

Lab — Shell Commands

LAB

```
# 1. Internal vs external commands
type ls         # external
type cd         # built-in

# 2. Manual pages
man grep
man 5 passwd    # section 5 = file formats

# 3. History
history | tail -20
!42             # re-run command 42
!!              # re-run last command
Ctrl+R          # reverse history search

# 4. Multi-output pipeline
cat /etc/passwd | cut -d: -f1,7 | sort | uniq

# 5. Grep with regex
grep -E "^[a-z]{4}:" /etc/passwd
```

Lab — sed, awk & Scripting

LAB

Write a System Information Script

```
#!/bin/bash
# sysinfo.sh — gather system information
set -euo pipefail

echo "=== SYSTEM INFO REPORT ==="
echo "Date:     $(date)"
echo "Hostname: $(hostname)"
echo "Kernel:   $(uname -r)"
echo "Uptime:   $(uptime -p)"
echo ""
echo "=== DISK USAGE ==="
df -h | awk 'NR==1 || /^\/dev/' | column -t
echo ""
echo "=== MEMORY ==="
free -h | sed -n '1,2p'
echo ""
echo "=== TOP 5 PROCESSES BY CPU ==="
ps aux --sort=-%cpu | awk 'NR<=6 {printf "%-10s %5s %5s %s\n", $1,$3,$4,$11}'
echo ""
echo "=== LOGGED IN USERS ==="
who | awk '{print $1}' | sort -u
echo "=== REPORT COMPLETE ==="
```

Save as `sysinfo.sh`, run with `chmod +x sysinfo.sh && ./sysinfo.sh`

Day 3 — Morning

Networking

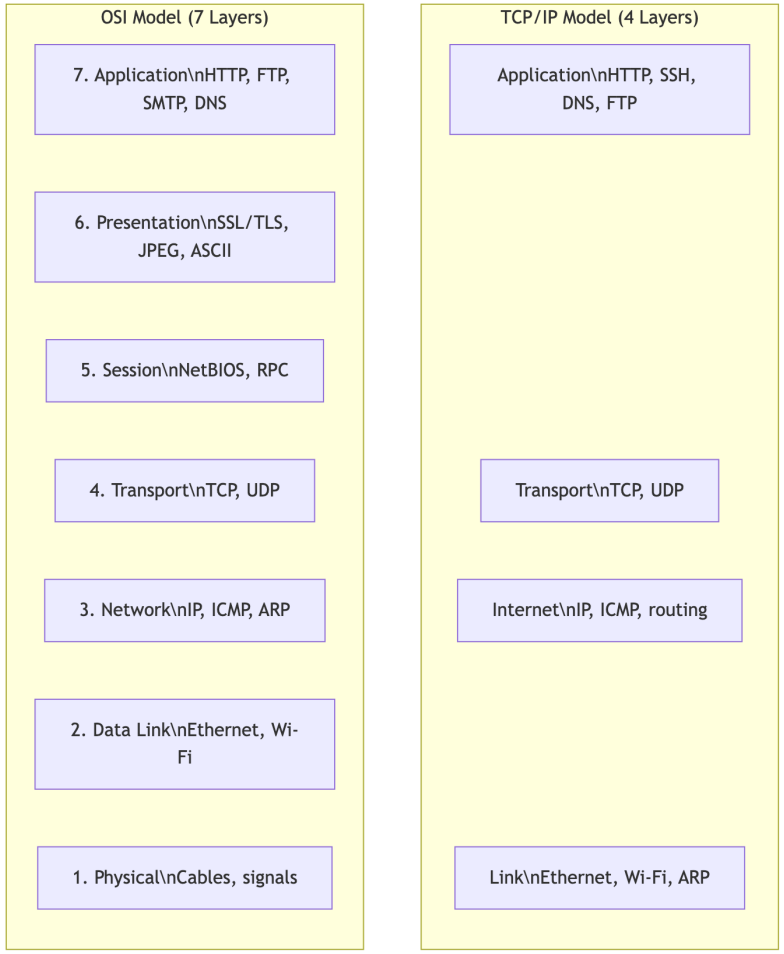
© Utopios Training

Agenda — Day 3 Morning

- OSI model vs TCP/IP model
- Network configuration under Linux
- DNS resolution process
- Diagnostics & troubleshooting methodology
- Firewall concepts (iptables, ufw)
- SSH in depth: config, tunneling, port forwarding
- File transfer and remote connections

© Utopios Training

OSI Model vs TCP/IP Model



Networking Concepts

Key Concepts

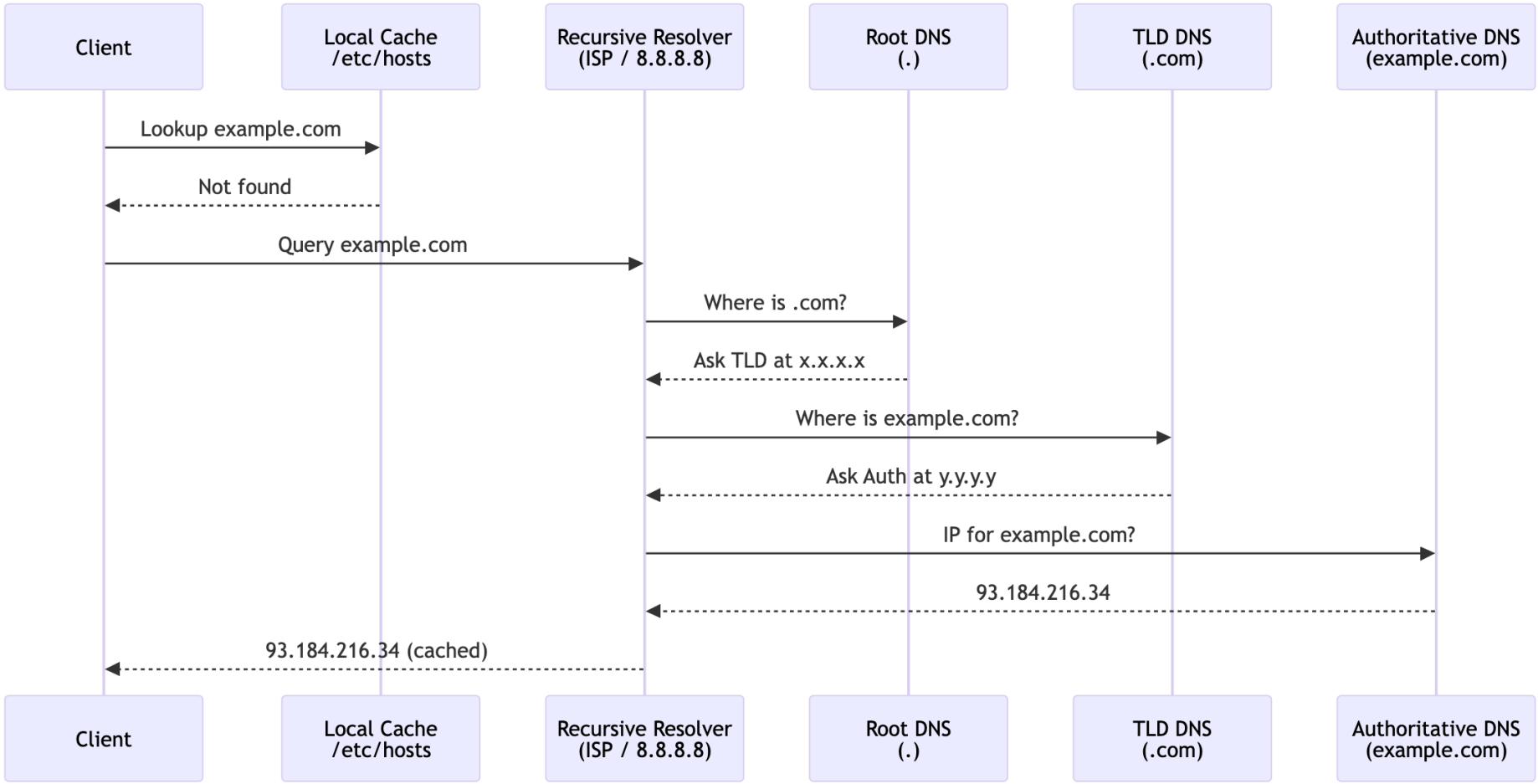
Concept	Description
IP address	Network layer identifier (IPv4/IPv6)
Subnet mask	Defines network / host portion
Gateway	Default router for external traffic
DNS	Domain Name System — name to IP resolution
DHCP	Dynamic IP address assignment
MAC address	Physical hardware identifier (Layer 2)

TCP/IP Model

```
Application  →  HTTP, SSH, DNS, FTP
Transport    →  TCP (reliable), UDP (fast)
Network      →  IP, ICMP, routing
Link         →  Ethernet, Wi-Fi
```

© Utopios Training

DNS Resolution Process



Linux DNS Configuration

```
# DNS resolution order (Name Service Switch)
cat /etc/nsswitch.conf
# hosts: files dns    → check /etc/hosts first, then DNS

# Static name resolution
cat /etc/hosts
# 127.0.0.1   localhost
# 192.168.1.50  myserver.local

# DNS resolver configuration
cat /etc/resolv.conf
# nameserver 8.8.8.8
# nameserver 1.1.1.1
# search example.com

# systemd-resolved (modern systems)
resolvectl status
resolvectl query example.com

# DNS lookup tools
dig example.com A              # detailed query
dig example.com MX             # mail servers
nslookup example.com           # simple lookup
host example.com               # compact output
```

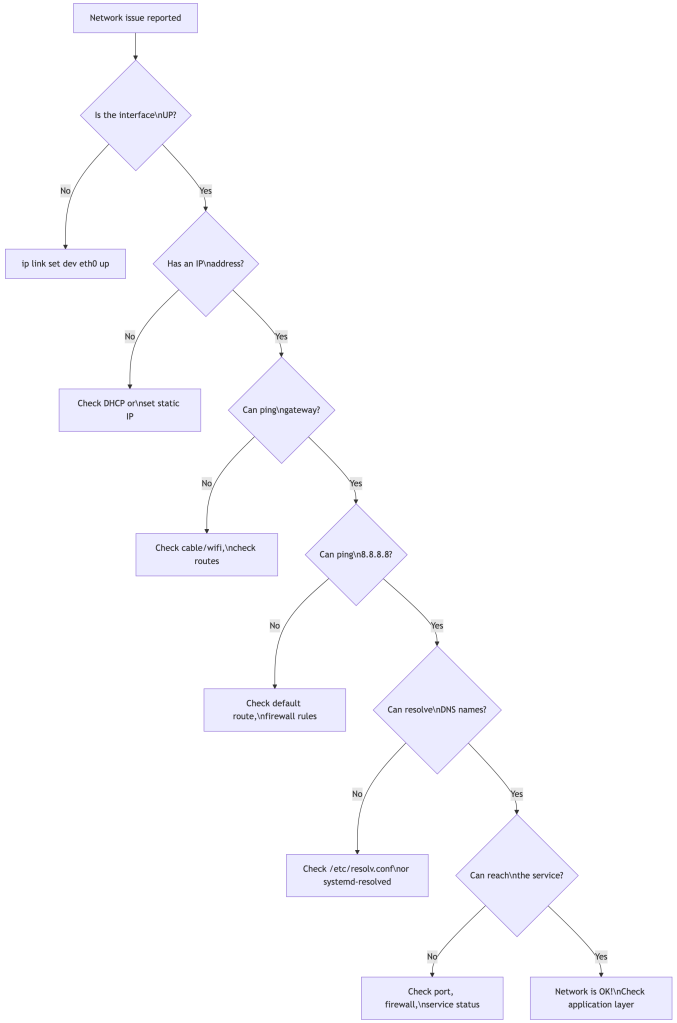
Network Configuration

```
# View interfaces
ip addr show          # modern (iproute2)
ip link show
ifconfig              # legacy (net-tools)

# Set IP address
sudo ip addr add 192.168.1.10/24 dev eth0
sudo ip route add default via 192.168.1.1

# DNS configuration
cat /etc/resolv.conf
cat /etc/hosts

# NetworkManager (desktop/server)
nmcli device status
nmcli connection show
nmcli con mod "Wired" ipv4.addresses 192.168.1.10/24
```

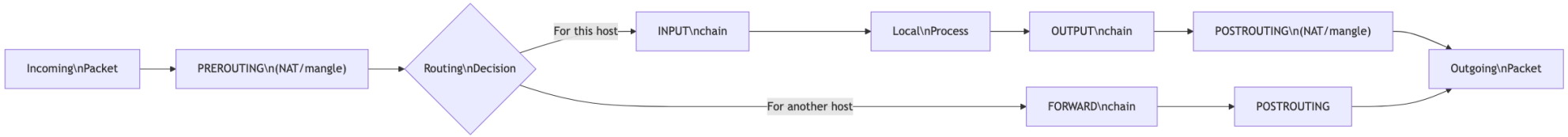
Diagnostics & Troubleshooting

```
# Connectivity
ping -c 4 8.8.8.8
ping6 ::1

# Routing
ip route show
traceroute 8.8.8.8

# DNS
nslookup google.com
dig google.com A
host google.com

# Open ports & connections
ss -tulnp                   # modern
netstat -tulnp              # legacy
lsof -i :80

# Firewall (basic)
sudo ufw status
sudo ufw allow 22/tcp
sudo iptables -L -n -v
```

Network Troubleshooting Methodology

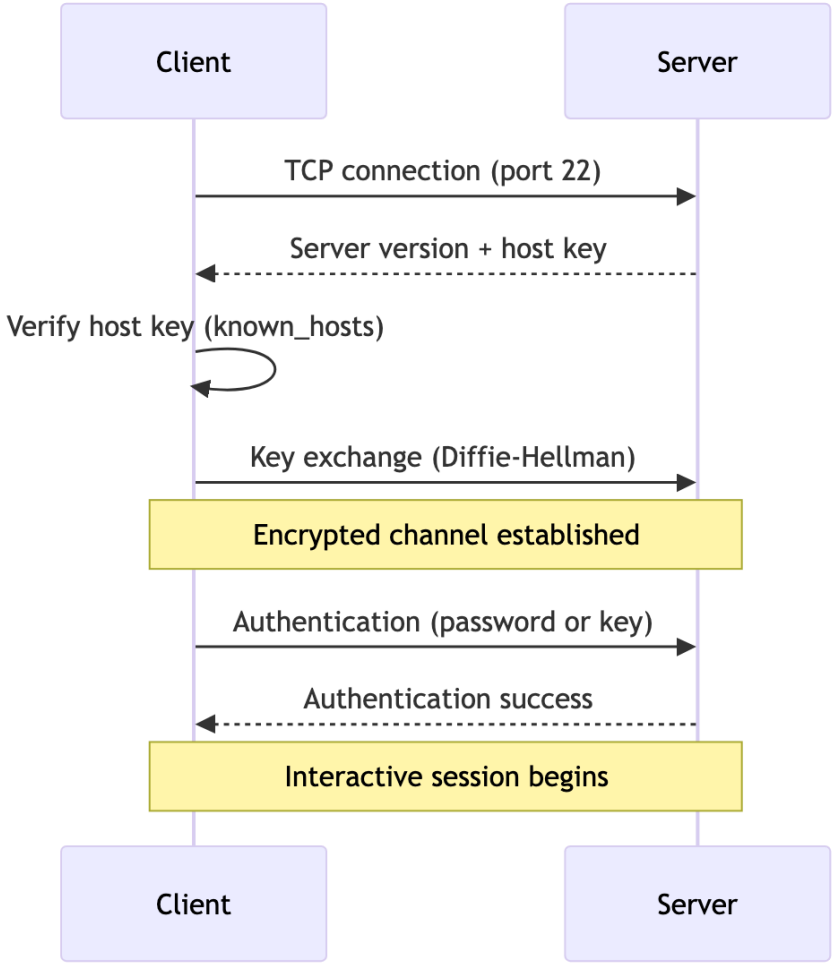


Firewall — iptables Chains




```
# iptables basics
sudo iptables -L -n -v                  # list rules
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT  # allow SSH
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT  # allow HTTP
sudo iptables -A INPUT -j DROP          # drop everything else

# ufw (simplified frontend)
sudo ufw enable
sudo ufw allow 22/tcp
sudo ufw allow from 192.168.1.0/24
sudo ufw deny 3306/tcp
sudo ufw status verbose
```

SSH — Secure Shell In Depth



SSH Configuration & Tunneling

```
# SSH config file (~/.ssh/config) — simplify connections
# Host myserver
#     HostName 192.168.1.100
#     User alice
#     Port 2222
#     IdentityFile ~/.ssh/id_myserver
# Then: ssh myserver (instead of ssh -p 2222 alice@192.168.1.100)

# Local port forwarding (access remote service via local port)
ssh -L 8080:localhost:80 user@server
# Now localhost:8080 → server's port 80

# Remote port forwarding (expose local service to remote)
ssh -R 9090:localhost:3000 user@server
# server:9090 → your localhost:3000

# Dynamic SOCKS proxy
ssh -D 1080 user@server
# Configure browser proxy: SOCKS5 localhost:1080

# SSH jump host (bastion)
ssh -J bastion@jump.host user@internal.host
```

© Utopios Training

File Transfer & Remote Access

```
# SSH - Secure Shell
ssh user@hostname
ssh -p 2222 user@server
ssh -i ~/.ssh/id_rsa user@server

# SSH key management
ssh-keygen -t ed25519 -C "comment"
ssh-copy-id user@server

# SCP - Secure Copy
scp file.txt user@server:/tmp/
scp -r directory/ user@server:/backup/

# rsync - efficient sync
rsync -avz local/ user@server:/remote/
rsync -avz --delete src/ dest/

# wget / curl
wget https://example.com/file.tar.gz
curl -O https://example.com/file.zip
curl -s https://api.example.com/data | jq .
```

© Utopios Training

Lab — Networking

LAB

```
# 1. Interface and IP information
ip addr show
ip route show

# 2. Test connectivity layer by layer
ping -c 3 127.0.0.1          # loopback
ping -c 3 $(ip route | awk '/default/{print $3}')  # gateway
ping -c 3 8.8.8.8            # internet
ping -c 3 google.com         # DNS resolution

# 3. DNS investigation
dig google.com +short
dig google.com MX +short
cat /etc/resolv.conf

# 4. Open ports
ss -tulnp

# 5. SSH key setup
ssh-keygen -t ed25519 -C "training"
cat ~/.ssh/id_ed25519.pub
```

Lab — Firewall & SSH

LAB

```
# 1. Check firewall status
sudo ufw status verbose  # Debian/Ubuntu
# or: sudo iptables -L -n

# 2. Configure basic firewall rules
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw enable

# 3. Test SSH config file
cat >> ~/.ssh/config << 'EOF'
Host testserver
    HostName 127.0.0.1
    User $USER
    Port 22
EOF
ssh testserver   # test connection

# 4. traceroute to understand routing
traceroute -n 8.8.8.8

# 5. Examine network connections
ss -s              # summary statistics
ss -tp             # TCP connections with process names
```

Day 3 — Afternoon

System Administration & Advanced Topics

© Utopios Training

Agenda — Day 3 Afternoon

- Package management (apt, yum/dnf, pacman)
- Standard streams: stdin, stdout, stderr
- Redirections in depth
- Cron jobs and task scheduling
- Log management (journalctl, /var/log, logrotate)
- Systemd service lifecycle
- System monitoring tools
- Disk management basics
- Final lab / mini-project
- Course summary and next steps

© Utopios Training

Package Management

Feature	Debian/Ubuntu	RHEL/Fedora	Arch
Tool	`apt`	`dnf` (yum)	`pacman`
Install	`apt install nginx`	`dnf install nginx`	`pacman -S nginx`
Remove	`apt remove nginx`	`dnf remove nginx`	`pacman -R nginx`
Update list	`apt update`	`dnf check-update`	`pacman -Sy`
Upgrade all	`apt upgrade`	`dnf upgrade`	`pacman -Syu`
Search	`apt search nginx`	`dnf search nginx`	`pacman -Ss nginx`
Info	`apt show nginx`	`dnf info nginx`	`pacman -Si nginx`
List installed	`dpkg -l`	`rpm -qa`	`pacman -Q`
Which pkg owns	`dpkg -S /bin/ls`	`rpm -qf /bin/ls`	`pacman -Qo /bin/ls`
Pkg format	`.deb`	`.rpm`	`.pkg.tar.zst`
Repos config	`/etc/apt/sources.list`	`/etc/yum.repos.d/`	`/etc/pacman.conf`

© Utopios Training

Package Management — Practical Examples

```
# Debian/Ubuntu (apt)
sudo apt update                     # refresh package index
sudo apt upgrade                    # upgrade all packages
sudo apt install -y vim curl git    # install multiple packages
sudo apt remove --purge nginx       # remove + config files
sudo apt autoremove                 # clean unused dependencies
apt list --installed | wc -l        # count installed packages

# RHEL/Fedora (dnf)
sudo dnf check-update
sudo dnf upgrade
sudo dnf install -y vim curl git
sudo dnf remove nginx
sudo dnf autoremove

# Check package before installing
apt policy nginx                    # show versions + repos
apt depends nginx                   # show dependencies
apt rdepends nginx                  # reverse dependencies
```

Standard Streams

```
┌──────────────┐
│              │ ← stdin  (fd 0)  keyboard / file / pipe
│   Process    │ → stdout (fd 1)  terminal / file / pipe
│              │ → stderr (fd 2)  terminal / file
└──────────────┘
```

```bash
# Demonstrate streams
echo "This goes to stdout"          # fd 1
echo "This is an error" >&2         # fd 2
read -p "Enter value: " VAR         # reads from fd 0

# /dev/null — the black hole
command > /dev/null 2>&1   # discard all output
command 2>/dev/null        # discard errors only
```

Redirection Operators

```
# Output
echo "hello" > file.txt        # overwrite stdout → file
echo "world" >> file.txt       # append stdout → file
ls /badpath 2> err.txt         # stderr → file
ls /badpath 2>> err.txt        # append stderr → file
ls /badpath &> all.txt         # stdout + stderr → file

# Input
sort < unsorted.txt            # stdin from file

# Here-document
cat << EOF > config.txt
host=localhost
port=3306
EOF

# Here-string
grep "pattern" <<< "inline string"
```

Pipes in Depth

```
# Simple pipe
ls -la | grep "^d"

# Multi-stage pipeline
cat /var/log/auth.log \
  | grep "Failed" \
  | awk '{print $11}' \
  | sort | uniq -c \
  | sort -rn \
  | head -10

# tee — split output to file AND stdout
ls -la | tee listing.txt | wc -l

# Process substitution
diff <(sort file1) <(sort file2)
```

Cron Jobs & Task Scheduling

```
# Crontab format:
# MIN  HOUR  DAY  MONTH  DOW  COMMAND
# 0-59 0-23  1-31 1-12   0-7

# Edit your crontab
crontab -e

# Examples
0 2 * * *     /home/user/backup.sh      # daily at 2:00 AM
*/15 * * * *  /usr/local/bin/check.sh    # every 15 minutes
0 9 * * 1     /opt/report.sh            # Mondays at 9:00 AM
0 0 1 * *     /opt/monthly-cleanup.sh   # first of each month

# List crontab
crontab -l

# System-wide cron
cat /etc/crontab
ls /etc/cron.d/
ls /etc/cron.daily/   # scripts run daily
ls /etc/cron.weekly/  # scripts run weekly

# Systemd timers (modern alternative)
systemctl list-timers --all
```

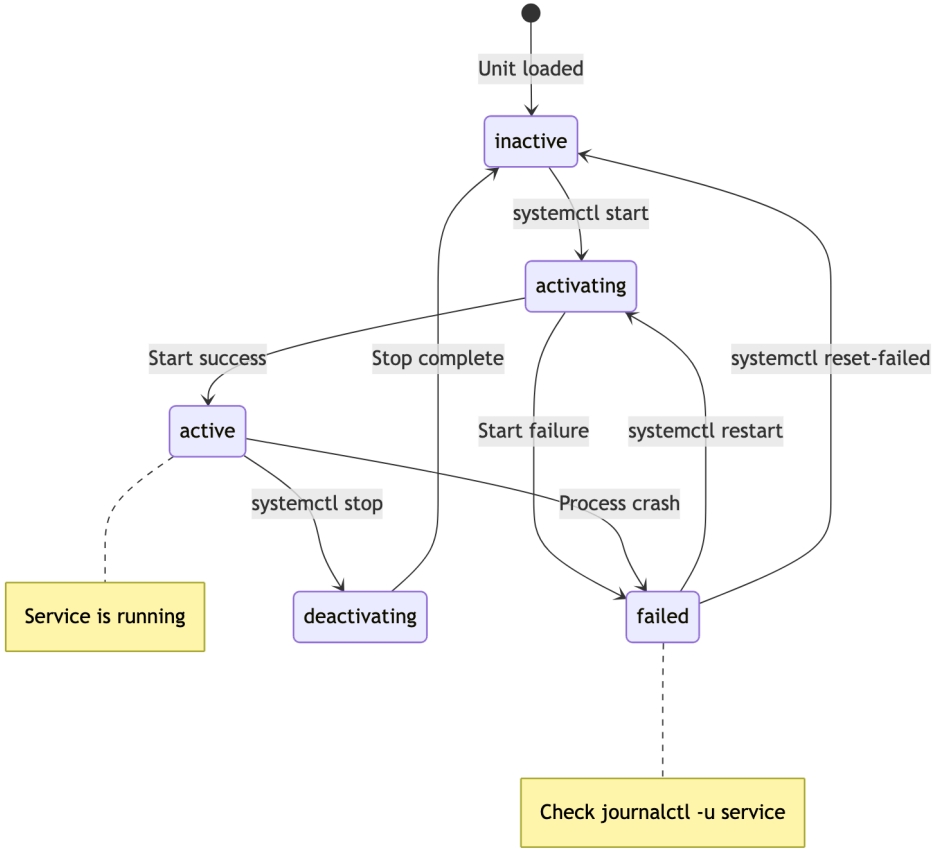
Cron — Special Strings & Tips

```
# Special time strings
@reboot     command    # at startup
@hourly     command    # every hour (0 * * * *)
@daily      command    # every day (0 0 * * *)
@weekly     command    # every week (0 0 * * 0)
@monthly    command    # every month (0 0 1 * *)
@yearly     command    # every year (0 0 1 1 *)

# Best practices for cron scripts
# 1. Always use absolute paths
0 2 * * * /usr/bin/python3 /home/user/backup.py

# 2. Redirect output (cron sends email otherwise)
0 2 * * * /opt/backup.sh >> /var/log/backup.log 2>&1

# 3. Use a lock file to prevent overlap
0 * * * * flock -n /tmp/myjob.lock /opt/hourly-job.sh

# 4. Set PATH at the top of crontab
PATH=/usr/local/bin:/usr/bin:/bin
```

Log Management

```
# Traditional logs in /var/log/
ls /var/log/
# syslog, auth.log, kern.log, dmesg, apt/, nginx/...

# journalctl (systemd journal)
journalctl                        # all logs
journalctl -u ssh                 # specific service
journalctl -f                     # follow (like tail -f)
journalctl --since "2024-01-01"   # by date
journalctl --since "1 hour ago"
journalctl -p err                 # only errors
journalctl -b -1                  # previous boot

# logrotate — automatic log rotation
cat /etc/logrotate.conf
ls /etc/logrotate.d/

# Example logrotate config:
# /var/log/myapp/*.log {
#     daily
#     rotate 14
#     compress
#     missingok
#     notifempty
#     create 640 root root
# }
```

Systemd Service Lifecycle



Systemd — Managing Services

```
# Service lifecycle commands
systemctl start nginx
systemctl stop nginx
systemctl restart nginx
systemctl reload nginx          # reload config without restart
systemctl status nginx

# Boot behavior
systemctl enable nginx          # start at boot
systemctl disable nginx         # don't start at boot
systemctl is-enabled nginx

# View service configuration
systemctl cat nginx             # show unit file
systemctl show nginx            # all properties
systemctl edit nginx            # override settings

# List services
systemctl list-units --type=service
systemctl list-units --type=service --state=failed

# Logs for a service
journalctl -u nginx --since "1 hour ago"
journalctl -u nginx -f          # follow
```

System Monitoring Tools

Tool	Purpose	Key Flags
`top`	Process viewer (built-in)	`top -o %MEM`
`htop`	Interactive process viewer	`htop -t` (tree)
`vmstat`	Virtual memory stats	`vmstat 1 5` (every 1s, 5x)
`iostat`	Disk I/O stats	`iostat -x 1`
`iotop`	Per-process I/O	`sudo iotop -o`
`free`	Memory usage	`free -h`
`sar`	System activity reporter	`sar -u 1 5` (CPU)
`nmon`	All-in-one monitoring	Interactive
`dstat`	Versatile resource stats	`dstat -cdnm`

```
# Quick system health check
uptime                    # load averages
free -h                   # memory
df -h                     # disk space
top -bn1 | head -20       # process snapshot
```

Disk Management Basics

```
# List block devices
lsblk
sudo fdisk -l

# Partition a disk (interactive)
sudo fdisk /dev/sdb       # MBR
sudo gdisk /dev/sdb       # GPT

# Create filesystem
sudo mkfs.ext4 /dev/sdb1
sudo mkfs.xfs /dev/sdb1

# Mount / unmount
sudo mkdir /mnt/data
sudo mount /dev/sdb1 /mnt/data
sudo umount /mnt/data

# Persistent mount (/etc/fstab)
# /dev/sdb1  /mnt/data  ext4  defaults  0  2
# UUID=xxxx  /mnt/data  ext4  defaults  0  2

# Find UUID
sudo blkid /dev/sdb1

# Check and repair filesystem
sudo fsck /dev/sdb1          # only on unmounted FS!
sudo e2fsck -f /dev/sdb1     # ext4 specific
```

Functions & Complete Script

```
#!/bin/bash
set -euo pipefail   # strict mode

# Constants
LOG="/var/log/myscript.log"

# Function
log() {
  echo "[$(date +%T)] $*" | tee -a "$LOG"
}

backup_dir() {
  local src="$1"
  local dst="$2"
  if [ ! -d "$src" ]; then
    log "ERROR: Source $src not found" >&2
    return 1
  fi
  rsync -aq "$src/" "$dst/"
  log "Backup of $src → $dst done"
}

# Main
backup_dir "/etc" "/backup/etc-$(date +%F)"
```

vi / vim for Script Writing

Essential Workflow

```
vim myscript.sh

# In normal mode:
i                   # enter insert mode
Esc                 # back to normal mode
:set number         # show line numbers
:syntax on          # syntax highlighting

# Navigation
gg / G              # go to first / last line
0 / $               # start / end of line
w / b               # next / prev word

# Editing
dd                  # delete line
3dd                 # delete 3 lines
u                   # undo   Ctrl+R redo
:%s/old/new/g       # global replace

:wq                 # save and quit
```

Lab — Redirections & Scripting

LAB

```
# 1. Redirect stdout and stderr separately
ls /etc /badpath > out.txt 2> err.txt
cat out.txt && cat err.txt

# 2. Merge streams
ls /etc /badpath &> combined.txt

# 3. Pipe chain
cat /etc/passwd | grep -v "^#" | cut -d: -f1,7 \
  | sort > users_shells.txt

# 4. Write a script with vi
vi backup.sh
# Add: #!/bin/bash
# rsync -av /home/ /tmp/home-backup/
chmod +x backup.sh
./backup.sh
```

Lab — Package Management & Cron

LAB

```
# 1. Package management
sudo apt update && sudo apt list --upgradable
apt search htop
sudo apt install -y htop tree ncdu
dpkg -L htop | head -10      # list files in package

# 2. Explore installed packages
dpkg -l | wc -l              # total installed packages
dpkg -l | grep -i "python"   # python-related packages

# 3. Cron job creation
crontab -e
# Add: */5 * * * * echo "$(date) - alive" >> /tmp/heartbeat.log
crontab -l

# 4. Verify cron is running
systemctl status cron
# Wait 5 minutes, then:
cat /tmp/heartbeat.log

# 5. Log exploration
journalctl --since "30 min ago" | tail -20
sudo cat /var/log/auth.log | tail -10
```

Lab — System Monitoring & Disk

LAB

```
# 1. Monitor system resources
free -h
vmstat 1 5
iostat -x 1 3

# 2. Use htop
htop    # press F5 for tree view, F6 to sort, q to quit

# 3. Disk usage analysis
df -h
du -sh /var/log/*  | sort -h | tail -10
sudo ncdu /var     # interactive disk usage (if installed)

# 4. Check fstab
cat /etc/fstab
# Identify: device, mount point, FS type, options

# 5. Explore block devices
lsblk -f
sudo blkid
```

Final Lab — Mini-Project

LAB

Build a Server Health Check Script

Create `/home/$USER/healthcheck.sh` that:

1. **Checks disk space** — warn if any partition > 80% used
2. **Checks memory** — warn if free memory < 20%
3. **Checks running services** — verify ssh and cron are active
4. **Lists failed systemd services**
5. **Reports top 5 CPU-consuming processes**
6. **Writes output to a log file** with timestamp
7. **Sends exit code 1** if any warning was triggered

© Utopios Training

Final Lab — Mini-Project

Bonus

- Add a `--verbose` flag for detailed output
- Add a cron job to run it every hour
- Use functions for each check
- Color output (green = OK, red = WARNING)

© Utopios Training

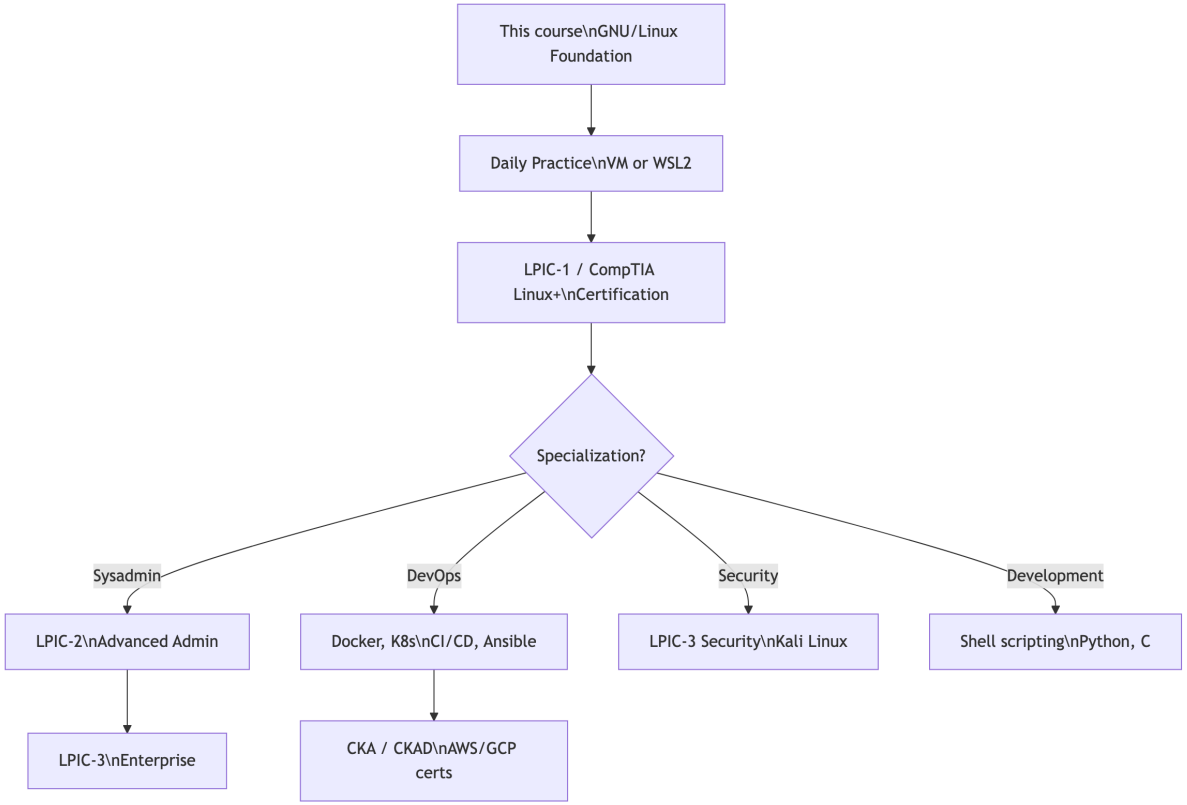
Course Summary

Key Commands Cheat Sheet

Task	Command
System info	`uname -a`, `cat /etc/os-release`
Users	`useradd`, `usermod`, `passwd`, `id`
Files	`ls`, `cp`, `mv`, `rm`, `find`, `file`
Permissions	`chmod`, `chown`, `getfacl`, `setfacl`
Text	`grep`, `sed`, `awk`, `cut`, `sort`, `uniq`
Processes	`ps`, `top`, `kill`, `systemctl`
Network	`ip`, `ss`, `ping`, `dig`, `ssh`, `curl`
Packages	`apt` / `dnf` / `pacman`
Logs	`journalctl`, `tail -f`, `logrotate`
Disks	`df`, `du`, `lsblk`, `fdisk`, `mount`
Scheduling	`crontab -e`, `systemctl list-timers`
Scripting	`bash`, `set -euo pipefail`, `trap`

© Utopios Training

Recommended Learning Path



Thank You!

GNU/Linux Foundation Training

© Utopios Training